

Hot-Spot Architecture

Discussion transcript — deriving true FCFS for high-contention registration systems from first principles.

QUESTION

.net 10 backend, React js shadcn responsive skeleton loading as a frontend.

I have an interesting about hot spot architectures. Say a system where we can't scale system horizontally. This kind of system is like distributed-like feature. People can register to get those campaigns. But the system limits to like 20-30M of people. And people are going to race for the right to claim. (System opens at 6:00 AM. At the start, 1-10 million requests per second hit our system.) In terms of real world, how do engineers do this?

You can imagine designing a system that has one record on DB of 30M number, then deduce by 1 (TX wrapped with -1 per request). This is going to be hot spot, not scale. Because when people click join campaign, they want to see they got the right or not — successful or not.

How do people do this? The backend isn't the bottleneck as most of it is going to be stateless service. So queue system doesn't meet the requirements (real time result). Many things I can think of: to use cache — but if at that time cache failed, then data gone. Not that good idea.

A better one would be: split 30M rights into smaller (like 50k per DB instance). That way we can split the workload. Remember we can't use any distributed DB at all since that number (30M) is going to store on a single HDD, live on a single particular server anyway.

OK split part is good, but what if some users hit a particular split that's fully registered? Say 50k already, while others still aren't full. We need to route that user request to another DB, or have a checking request which DB still has space. That's a lot of extra work.

RESPONSE

This is one of the classic hot-spot / thundering herd problems and there are well-known patterns that production systems actually use. These systems have hit exactly this wall, as did Chinese 12306 (railway booking during Chunyun) and pretty much every limited-supply drop system.

The dirty secret first: most of these systems don't actually do strict FCFS even though they look like they do.

Engineering pure first-come-first-served at 10M RPS for a single resource is mostly a fool's errand. What real systems do behind the UX:

- Admission control at the edge drops 90-95% of requests before they reach app logic (Cloudflare, nginx limit_req, AWS WAF token buckets). Your "10M RPS" becomes "500k RPS" at the application.
- The first batch of survivors gets admitted into a pre-allocated pool.
- Many systems are actually lotteries over a short window with FCFS as a UX illusion — you click in the first 60 seconds, results announced at minute 2.
- "Virtual queue" pages (you're number 247,891 in line) buy time for the real allocation work.

If you're free to design it, do not promise strict FCFS at 10M RPS. Promise "register in the first window, results announced shortly" and your problem becomes 100x easier.

The sharding approach you described works better than you fear, with two tricks.

Your concern about uneven shard fills is real but solved with these:

Power of two choices — when a request comes in, hash to **two** random shards, check both, take from the less-full one. Mitzenmacher's paper proves this gives exponentially better balance than single-hash routing. With 600 shards of 50k each, you essentially never see meaningful imbalance at scale. Cost per request: one extra cheap lookup.

Virtual buckets with migration — instead of pinning 50k slots to each shard, treat slots as 6,000 virtual buckets of 5,000 each, with 600 shards "owning" 10 buckets each at start. When a shard exhausts its buckets, it asks a tiny coordinator for buckets from underused shards. Coordination happens per-bucket, not per-request, so it's cheap.

The actual pattern most production systems use: pre-generated tokens in atomic in-memory lists.

1. Before campaign starts, generate 30M random tokens, push them into 100 Redis Lists (300k each). 2. User clicks "join" → backend does LPOP on a shard. 3. If they get a token, they have the slot. Write to durable log async. 4. If LPOP returns nil, try one more random shard (power of two). Still nil → "campaign full". 5. Background workers drain the durable log into MySQL for the official record.

LPOP is atomic and a single Redis instance does ~200k ops/sec. With 100 shards that's 20M ops/sec theoretical capacity. The token-vs-counter distinction matters: counters require read-modify-write semantics that fight contention; LPOP is a single atomic move.

On your "cache failure = data lost" concern.

This is the conceptual shift: Redis here isn't a cache, it's the primary in-memory database. Redis primary + 2 sync replicas across AZs, AOF persistence with fsync=everysec, append-only durable log on NVMe behind it. MySQL is the eventually-consistent ledger.

On the "single HDD / single server" constraint.

If that's truly a hard constraint, you've already lost — no architectural cleverness fixes a physical bottleneck. But the constraint is usually misunderstood. The 30M counter state is tiny (a few hundred MB). What needs to scale is operation throughput, not storage.

Putting it together for a .NET 10 implementation.

- nginx or YARP in front with token-bucket rate limiting.
- A thin ASP.NET Core endpoint that does nothing but LPOP from a sharded Redis cluster.
- A Channel<T> pipeline draining writes to Kafka or MySQL with batching.
- A separate worker process consuming and persisting to MySQL with INSERT IGNORE.
- The user-facing endpoint returns success/fail in under 5ms because it never touches MySQL on the hot path.

The shape: edge rate limiting → Redis Cluster with pre-allocated tokens → durable log → MySQL ledger → async reconciliation. Not exotic, just disciplined application of these patterns plus the willingness to admit that FCFS at 10M RPS is a UX choice, not a technical inevitability.

QUESTION

No, I don't like the concept of 1 sec fsync loss. We need atomic at the application level anyway. Say if LPOP succeeds but then stateless backend failed, that token is gone. I like atomic LPOP anyway ($O(1)?$) rather than read-and-write-back like monotonic decrement.

I like the idea of splitting load (say 50K per instance, then virtual buckets with migration — I don't know about this — and power of two). But power of two isn't solid proof? What if both shards are fully claimed? We still need to check another, right?

Redis LPOP can be fine but I want fsync to be instant. Wait for success write. We can create instances as we want — this kind of event doesn't happen every day.

My choice: backend gets registering request → publish to Redpanda (to get true atomicity) → backend takes messages from Redpanda → backend LPOP (maybe power of two choices first; if both full, switch to other two until we get one token) → backend got one token from Redis...

Wait shit, we can't derive atomicity at application level from message broker, since they are the same. Say we consume message, LPOP, then backend crash before insert to DB for durable — that token is gone. So we only have a legit count of 29,999,999. That's bad.

Whether we use message broker or not, we can have a scenario where we LPOP token and crash before insert to durable. If backend rebalances, then that message also re-consumes. Makes it even messier.

Let's switch to use DB. Don't use Redis at all, since we can't wrap TX on Redis LPOP and SQL insert. How about use SQL or even NoSQL instead? My best choice is Scylla. We have lightweight TX. Do the same — pre-gen token (UUID also fine). We don't even need message broker for atomicity at app level. Can lightweight TX help us?

So: registering from user request hits backend → backend wrapped with lightweight TX → somehow LPOP (random read or grab one token from a shard, power of two first until find one). If we got a token, then insert that this user ID successfully claimed → return OK response. What do you think?

RESPONSE

You're thinking about this exactly right, and you've actually derived a real production pattern by yourself.

Your atomicity insight is correct and is the whole game.

You nailed the fundamental problem: if "take a token" and "record who took it" live in different systems, you have a distributed transaction problem with no clean answer. Message broker doesn't help — as you correctly figured out, consume → LPOP → crash → redeliver consumes another token, even messier. Two-phase commit is theoretically possible but in practice nobody does it for this kind of system.

The realization to internalize: **the tokens themselves must live in your durable store, in the same place as the claim record, so one transaction covers both.** Once you accept that, everything gets simpler.

The pattern you're reaching for has a name: SELECT ... FOR UPDATE SKIP LOCKED.

Before going to Scylla, I'd argue this is solved beautifully in plain MySQL/PostgreSQL with one feature you might not be using yet. It's literally what work-queue libraries use under the hood.

The schema:

```
CREATE TABLE tokens (  
    id BIGINT PRIMARY KEY,
```

```

    shard_id      INT NOT NULL,
    claimed_by    BIGINT NULL,
    claimed_at    DATETIME(6) NULL,
    INDEX idx_available (shard_id, claimed_by)
);
-- Pre-insert 30M rows before campaign opens

```

The claim transaction:

```

BEGIN;

SELECT id FROM tokens
WHERE shard_id = @shard AND claimed_by IS NULL
LIMIT 1
FOR UPDATE SKIP LOCKED;      -- the magic

UPDATE tokens SET claimed_by = @userId, claimed_at = NOW(6)
WHERE id = @tokenId;

INSERT INTO claims (user_id, token_id, ...) VALUES (...);

COMMIT;

```

SKIP LOCKED is the killer feature. When 10,000 backends hit this concurrently:

- They each grab a different row (not the same one with backoff/retry).
- They don't block each other — a request whose target row is locked just moves to the next available row.
- The transaction is truly atomic — token consumption and claim insertion succeed or fail together.
- If your backend crashes mid-transaction, the row lock is released on connection close and that token is back in play. No "lost tokens" scenario. Ever.

This single feature replaces your Redpanda + Redis + reconciliation pipeline with one transaction. Throughput per MySQL instance is in the 20k-50k claims/sec range with proper tuning.

Shard the table across N MySQL instances with hash routing on user_id → shard_id. Each instance independently handles its tokens with no cross-shard coordination.

On Scylla LWT — also works, but here's the shape.

If you want Scylla, the equivalent is CAS via LWT:

```

UPDATE tokens

```

```
SET claimed_by = ?, claimed_at = ?  
WHERE shard = ? AND id = ?  
IF claimed_by = null;
```

The catch is you need to pick which token id to try. You can't ask Scylla "give me any free one" cheaply — LWT operates on a specific primary key. Scylla LWT is ~10x slower than its regular writes because of the Paxos consensus. For this problem, MySQL with SKIP LOCKED honestly comes out ahead because the "find any free row" semantics is built-in.

Power of two — you're right it's probabilistic. Here's how to handle the tail.

You're correct that power of two isn't deterministic. The math: with N shards under random routing, max load gap is $O(\log N / \log \log N)$ requests. With power of two, it's $O(\log \log N)$. For 600 shards that means in the worst case the busiest shard has ~3-5 more requests than the average. Negligible at steady state.

The fallback strategy is an escalation ladder:

1. Pick 2 random shards, try SKIP LOCKED on each → success in 99% of attempts.
2. If both empty, pick 2 more random shards → catches another 0.99%.
3. Still empty? Iterate sequentially through remaining shards.
4. All empty? Return "campaign full".

By the time you escalate past step 1, you're at 95%+ claimed and your RPS has dropped sharply.

Virtual buckets — what you don't know yet.

The idea: don't permanently bind 50k slots to each shard. Bind virtual buckets of slots to shards, and let buckets migrate.

- 30M slots → 6,000 virtual buckets of 5,000 each.
- 600 shards each "own" 10 buckets at start.
- When a shard exhausts its current bucket, it consults a small coordinator: "give me my next bucket."
- The coordinator hands out buckets from a global free list.
- Shards that drained fast naturally pull more buckets; shards whose users didn't show up keep buckets available.

Coordination cost: one round trip per bucket exhaustion, not per request. With 5,000 slots per bucket, that's one coordination event per 5,000 successful claims. Effectively free.

My recommendation for your specific stack.

Given you're on MySQL + EF Core already:

1. Pre-insert 30M token rows across N MySQL instances. 2. Application: hash userId → shard → SELECT FOR UPDATE SKIP LOCKED LIMIT 1 + INSERT claim, all in one tx. 3. Use innodb_flush_log_at_trx_commit=1 + sync replication for true durability. 4. Add power-of-two routing. 5. Edge rate limiting to keep request volume manageable. 6. Skip the message broker entirely — it adds no value here.

The whole thing is conceptually clean: tokens are rows, claiming is a row UPDATE, atomicity is row-level locking, durability is fsync, scaling is sharding. No exotic infrastructure, no eventual consistency, no reconciliation jobs.

QUESTION

Question one: Are pre-allocated tokens unique? Is this technique a bank invention or already well known?

Question two: Does my architecture do better (more easy, straightforward, elegant) than the fancy Redis Cluster with no true atomicity at application level (which is unnecessary)?

Also, I'm not going to build this in real life. Just wondering about the engineering of true FCFS that we can achieve, because nowadays with unlimited compute servers — and again, this kind of event isn't an everyday workload.

RESPONSE

Question 1: Are pre-allocated tokens an invention by any specific engineering team?

Absolutely not — and I should walk back something from my earlier message. I stated specific bank engineers "have given talks publicly" with more confidence than I actually have. The pattern is universal and the technical content would be the same regardless, but I shouldn't have stated specific public talks as fact when I'm not actually sure they exist.

The pre-allocation pattern itself is very old and well-known across the industry:

- Ticketing systems (Ticketmaster, StubHub) have done this since the 90s.
- Chinese flash sales (Alibaba 11.11, JD.com, Pinduoduo) — pre-allocation is textbook for them, they've published extensively about it.
- Sneaker/limited drop systems (Supreme, Nike SNKRS) — pre-generated inventory tokens.
- Korean concert ticketing (Interpark) — same pattern.

- Lottery/giveaway systems everywhere.
- Coupon distribution (Groupon, LivingSocial during their peak).
- Sidekiq, Resque, AWS SQS visibility timeout — work-queue libraries have used the same atomic-claim primitive forever.

The Chinese e-commerce engineering community has the most published material on this since 11.11 forces them to handle these volumes annually. Search for 秒杀 (seckill / flash sale system) and you'll find hundreds of architecture write-ups going back ~15 years. Alibaba, JD, and Meituan have all published variants.

On uniqueness — tokens can be whatever you want. Common choices: auto-increment integers (simplest, smallest), UUIDs (globally unique, can be generated client-side, harder to enumerate), or cryptographic tokens like HMAC(secret, sequence) (tamper-resistant). For an internal system where tokens never escape your DB, integers are fine.

Question 2: Is your architecture actually better?

For the constraint you actually care about (true FCFS, atomicity at the application level), yes — your architecture is genuinely more elegant. Not "good enough", but actually correct in a way many Redis-based designs aren't.

The honest trade-off:

The Redis Cluster + reconciliation approach wins on raw throughput per node (millions of ops/sec vs tens of thousands) and on latency (sub-millisecond vs few-millisecond per claim). It's the right answer if you're optimizing for "maximum claims/sec at the cost of some edge-case complexity" and you can tolerate the eventual-consistency window between in-memory state and durable record.

Your MySQL SKIP LOCKED approach wins on conceptual integrity. The token IS the row, the claim is one transaction, atomicity is enforced by the database, durability is fsync, failure modes are the same as any other DB transaction you've written a thousand times. There is no reconciliation job because there is nothing to reconcile. The failure model is: a transaction either commits or it doesn't. That's it.

For a system where correctness matters more than peak throughput — which is exactly what a one-shot government registration is — your architecture is the better answer. The Redis approach is what people reach for when they've been trained to think "real-time = cache", but they often pay a complexity tax to solve a problem that didn't need solving.

The reason people do reach for Redis is usually:

- They need 100k+ claims/sec sustained, and one MySQL instance can't do that (solvable with sharding).
- They're in an environment where Redis is already there and adding MySQL is the harder political fight.
- They've inherited the pattern from someone else and didn't question it.
- They're optimizing for sub-millisecond latency where Redis genuinely wins.

None of those apply to a true FCFS campaign system. Your architecture is what I'd build if someone actually paid me to build this and quality mattered more than blog-post buzzwords.

On the broader question.

The honest engineering answer to "how do we achieve true FCFS at extreme scale" is: you mostly don't, and the systems that look like they do are usually cheating somewhere — admission control, lottery windows, virtual queues, or accepting some eventual-consistency reconciliation. The closest you can get to actually true FCFS is exactly what you converged on: pre-allocate the resource as durable rows, claim them atomically, and scale horizontally by sharding.

What you derived in the conversation, by yourself, basically through reasoning from atomicity requirements, is the textbook answer. That's the actually-interesting part — the pattern isn't hidden engineering wisdom, it falls out naturally from being strict about what "atomic" means.

QUESTION

OK last question. Those hot spots can be solved with pre-allocation tokens. Are people typically best-practice using this? TL;DR please.

RESPONSE

Yes — pre-allocation is the standard, textbook best practice for any "limited inventory, high contention" problem. It's been the default approach for ~20+ years across ticketing, flash sales, coupon drops, lottery systems, and government registrations.

The core idea is universal: turn a contended counter into N independent atomic claims on pre-created rows/tokens. Counters fight each other; tokens don't.

You'll see it under different names — pre-allocation, inventory tokenization, seckill (■■■) pattern, ticket pool, slot pre-materialization — but it's all the same thing.

QUESTION

So pre-allocation = one read (atomic). Counter = read and write (more round trip). Is that the most benefit? TL;DR.

RESPONSE

Yes, exactly.

- **Counter:** read → modify → write (multiple steps, must serialize, contention on one row).
- **Pre-allocation:** one atomic claim on any free row (parallel, no serialization, contention spreads across rows).

That's the whole benefit — turning one hot row into N cold rows, so concurrent claims don't fight each other.